E-Signature Process Overview

Current Flow (Document Preparation)

1. Documents Upload
2. Recipient Add
3. Signature Fields
4. Security
5. Settings
6. Review

Pawnesh

Updated Flow (Document Preparation)

1. Document Upload
2. Choose Path
    a. Path 1: Self Service (skips Recipient Add)
    b. Path 2: Recipient Based (follows current flow Steps 2–6)
3. Form Creation (Path 1)
4. Bind Form with Self-Service Envelope
5. Embed Form (on Website or Share Form Link)

Current Signing Flow

1. Follow the link received via email.
2. Click on the Sign Field and draw or upload the signature.

Upgraded Signing Flow

Step 1: Access the document link via Email, Embedded Website Form, or other source.

Step 2: Pre-Sign Security & Signature Processing

Step	Action	Description / Notes
2.1	OTP Challenge	Email / SMS / Phone verification
2.2	Prepare Signature Placeholder	System adds placeholder in PDF and computes exact byte-range hash
2.3	Generate Cryptographic Signature	X.509 certificate + CMS format

2.4	Timestamping (Optional)	Trusted timestamp token (TSA) added for non-repudiation
2.5	Save Signed PDF & Progress Envelope	Envelope moves to next signer in order
2.6	Evidence / Audit Storage	Store docHash, certSerial, actor, timestamp, OTP result, IP/Device info, optional anchorTx / Merkle proof
2.7	Validation Endpoint	Reconstructs and verifies all stored evidence for compliance

---

Layered Flow of the Enhanced Digital Signature System

Step	What happens	Libraries involved	Why / Purpose
Upload Document / Create Envelope	User uploads PDF; empty envelope is created in DB	None at this step	Existing app logic; sets up envelope and tracks metadata
Add Recipients	Define recipients, roles, order of signing	None directly	Existing logic to handle users and permissions
Place Signature Fields	Map fields (signature, initials, date, etc.) on the PDF	**pdf-lib**	Prepares the PDF for signing; adds visible signature fields for each recipient
Send Envelope	Email notification sent to recipients	None directly	Existing mail service triggers; no crypto needed yet
Recipients Sign	Recipients click in-field to sign PDF	**node-signpdf**, **jsrsasign**, **speakeasy / otplib**	Node-signpdf applies digital signature to PDF; jsrsasign adds timestamp (RFC 3161); speakeasy/otplib handles OTP/TAN for strong authentication
Envelope Completed	All recipients sign → envelope status updated	**winston / pino**	Write immutable audit trail logs; ensures compliance

Crypto Layer (Security & Signing)

Step	What happens	Libraries	Why
Key & Certificate Generation	Generate RSA/ECDSA keys, X.509 certs	**node-forge**, **PKI.js**	Node-forge: create dev/test certs and serial numbers; PKI.js: parsing & validation of certs for compliance
PDF Signing	Apply signature to PDF	**node-signpdf**	Signs the document in a compliant manner (SES/AdES)
Timestamping	Record signing time for long-term validation	**jsrsasign (TSP)**, optionally **OpenTimestamps**	Ensures the signature is tamper-evident and legally valid
OTP/TAN Verification	Ensure signer is authenticated	**speakeasy / otplib**	Provides second-factor authentication; satisfies AES compliance in EU/UK and IT Act in India

Infra Layer (Audit, Hashing, Anchoring)

Step	What happens	Libraries	Why
Hash Document	Generate SHA-256/512 of signed document	**hasha**, **Node crypto**	Creates immutable hash for auditing and anchoring
Merkle Root Creation	Batch multiple document hashes	**merkletreejs**	Efficient way to anchor multiple documents at once
Blockchain Anchoring	Store Merkle root or hash on Ethereum / Bitcoin	**ethers.js / web3.js**, **bitcoinjs-lib**	Provides immutable, tamper-proof proof of signing for legal compliance
Audit Trail Logging	Log all events and signing metadata	**winston / pino**	Structured, append-only logs to comply with US ESIGN, EU eIDAS, India IT Act, etc.

How it differs from your current flow

Current Flow	Enhanced Flow with Libraries	Benefits
Users upload PDF → add recipients → recipients sign → envelope completed	Same basic flow, but **PDF signed digitally, timestamps added, hashes anchored, OTP verification**, audit logs written	Legal compliance (SES, AdES, AES), tamper-proof signatures, stronger authentication, future LTV support